

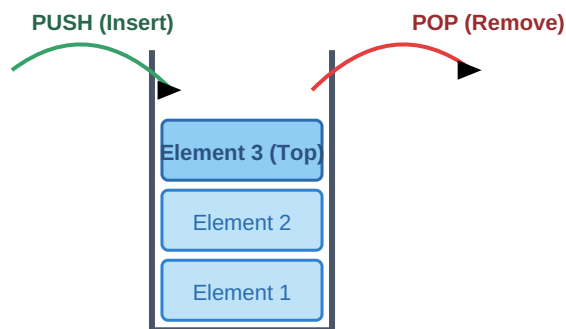
Data Structures Practice Questions

Question 1: Explain Stack using LIFO concept with a diagram.

A **Stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle. This means that the element inserted last into the stack will be the first one to be removed. It behaves similarly to a real-world stack of plates, where you can only add a new plate to the top or remove the topmost plate.

There are two primary operations associated with a stack:

- **Push:** Adds (inserts) an element onto the top of the stack.
- **Pop:** Removes (deletes) the top element from the stack.



Question 2: What is the difference between Stack and Queue?

While both are linear data structures used to store collections of elements, they differ primarily in the order in which elements are accessed and removed:

Feature	Stack	Queue
Core Principle	LIFO (Last In, First Out) – The last element added is the first to be removed.	FIFO (First In, First Out) – The first element added is the first to be removed.
Pointers / Access	Uses a single pointer called Top to monitor both insertions and deletions.	Uses two pointers: Front (for deletion) and Rear (for insertion).
Operations	Push (insertion) and Pop (deletion).	Enqueue (insertion) and Dequeue (deletion).
Real-world Analogy	A stack of dinner plates or an undo/redo history stack.	A queue of people waiting in line at a checkout counter.

Question 3: Write C code for Enqueue and Dequeue operations.

Below is a complete, compilation-ready C code snippet demonstrating standard queue operations (Enqueue and Dequeue) utilizing an array-based structure implementation with static bound limits:

```
#include <stdio.h>
#define SIZE 5

int queue[SIZE];
int front = -1;
int rear = -1;

// Function to insert an element into the queue
void enqueue(int value) {
    if (rear == SIZE - 1) {
        printf("Queue Overflow! Cannot enqueue %d", value);
    } else {
        if (front == -1) {
            front = 0; // Initialize front pointer on first insertion
        }
        rear++;
        queue[rear] = value;
        printf("Enqueued: %d", value);
    }
}

// Function to remove an element from the queue
void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow! No elements to dequeue");
    } else {
        printf("Dequeued: %d", queue[front]);
        front++;

        // Reset the queue pointers when it becomes empty
        if (front > rear) {
            front = rear = -1;
        }
    }
}

int main() {
```

```
// Demonstration of standard Queue operations
enqueue(10);
enqueue(20);
enqueue(30);

dequeue();
dequeue();

return 0;
}
```